

Laravel Bella Vista — Full Project Reference

Your personal cheatsheet from building the Bella Vista restaurant website

Is this MVC?

Yes. Laravel is an MVC framework. Here's exactly how it maps:

```
Model      →  app/Models/          (talks to the database)
View       →  resources/views/      (what the user sees)
Controller →  app/Http/Controllers (handles logic between M and V)
```

The flow for every request:

```
Browser Request
  ↓
routes/web.php      (which URL goes where?)
  ↓
Controller          (what logic runs?)
  ↓
Model / Eloquent    (get/save data from DB)
  ↓
Blade View          (render HTML with data)
  ↓
Browser Response
```

Folder Structure

```
myapp/
├─ app/
│  └─ Http/
│     └─ Controllers/      ← your controllers live here
│        ├── MenuController.php
│        ├── ReservationController.php
│        └── AdminController.php
│     └─ Auth/             ← auto-generated by laravel/ui
│        └─ Middleware/    ← your middleware lives here
│           └─ AdminMiddleware.php
```

```
| └─ Models/                                ← your models live here
|     └─ Category.php
|     └─ MenuItem.php
|     └─ Reservation.php
|
| └─ database/
|     └─ migrations/                        ← one file per table
|     └─ seeders/                          ← fake data files
|
| └─ resources/
|     └─ views/                            ← all blade templates
|         └─ layouts/
|             └─ app.blade.php             ← master layout
|         └─ home.blade.php
|         └─ menu/
|             └─ index.blade.php
|         └─ reservations/
|             └─ create.blade.php
|     └─ admin/
|         └─ dashboard.blade.php
|         └─ menu.blade.php
|
| └─ routes/
|     └─ web.php                            ← all your routes
|
| └─ bootstrap/
|     └─ app.php                            ← middleware registration
|
| └─ .env                                    ← DB credentials (never push to git)
```

Phase-by-Phase Summary

Phase 1 — Setup

- Installed Laravel via Composer
- MAMP provides MySQL, `php artisan serve` runs the web server
- `.env` file connects Laravel to the database
- Key commands:

```
bash
```

```
php artisan serve          # start dev server
php artisan config:clear   # clear cached config (run after .env changes)
```

Phase 2 — Migrations

- Migrations = PHP version control for your database
- Each migration has `up()` (create) and `down()` (rollback)
- Key commands:

```
bash

php artisan make:migration create_X_table
php artisan migrate                # run all pending migrations
php artisan migrate:rollback      # undo last batch
```

Phase 3 — Models & Eloquent

- Each model = one table
- `$fillable` protects against mass assignment attacks
- Relationships defined as methods
- Key commands:

```
bash

php artisan make:model ModelName
php artisan tinker          # interactive console to test queries
```

Phase 4 — Controllers & Routes

- Controllers handle request logic
- Routes in `web.php` map URLs to controller methods
- Named routes let you reference URLs by name instead of hardcoding
- Key commands:

```
bash

php artisan make:controller ControllerName
php artisan route:clear    # clear cached routes
php artisan route:list     # see all registered routes
```

Phase 5 — Blade Views

- Blade is Laravel's templating engine
- Master layout defined once, pages extend it
- Data passed from controller to view via `compact()`

Phase 6 — Admin Panel

- Auth system from `laravel/ui`
- Custom middleware protects admin routes
- Route groups apply middleware/prefix/names to multiple routes at once
- Key commands:

```
bash
```

```
composer require laravel/ui
php artisan ui bootstrap --auth
php artisan make:middleware MiddlewareName
```

Eloquent Cheatsheet

```
php
```

```
// CREATE
Category::create(['name' => 'Starters', 'slug' => 'starters']);

// READ ALL
Category::all();

// READ ONE
Category::find(1);
Category::where('slug', 'starters')->first();
Category::where('slug', 'starters')->firstOrFail(); // auto 404 if not found

// READ WITH CONDITIONS
MenuItem::where('is_available', true)->get();
MenuItem::where('price', '>', 10)->orderBy('price')->get();

// READ WITH RELATIONSHIP (eager loading)
Category::with('menuItems')->get(); // prevents N+1 problem
Category::with('menuItems')->find(1);

// UPDATE
$item = MenuItem::find(1);
$item->name = 'New Name';
$item->save();
// or:
MenuItem::find(1)->update(['name' => 'New Name']);

// DELETE
MenuItem::find(1)->delete();

// COUNT
Reservation::where('status', 'pending')->count();
```

Relationships

php

```
// ONE-TO-MANY (Category has many MenuItems)

// In Category model:
public function menuItems() {
    return $this->hasMany(MenuItem::class);
    // Laravel auto-uses category_id as FK
}

// In MenuItem model:
public function category() {
    return $this->belongsTo(Category::class);
    // Laravel auto-uses category_id on this table
}

// Usage:
$category->menuItems      // collection of menu items
$menuItems->category       // single category object
$menuItems->category->name  // chain through relationship
```

Convention Over Configuration

Laravel guesses a lot from naming conventions:

What you write	What Laravel infers
Model <code>Category</code>	Table <code>categories</code>
Model <code>MenuItem</code>	Table <code>menu_items</code>
Column <code>category_id</code>	FK to <code>categories.id</code>
Method <code>menuItems()</code>	Relationship uses <code>category_id</code>
<code>->constrained()</code>	References <code>categories(id)</code>

Golden rule: name things right, Laravel does the rest.

Migrations Syntax

php

```
$table->id(); // BIGINT UNSIGNED AUTO_INCREMENT PK
$table->string('name'); // VARCHAR(255) NOT NULL
$table->string('name')->nullable(); // VARCHAR(255) NULL
$table->string('slug')->unique(); // VARCHAR(255) UNIQUE
$table->text('description'); // TEXT
$table->decimal('price', 8, 2); // DECIMAL(8,2) e.g. 999999.99
$table->boolean('is_available'); // TINYINT(1)
$table->integer('order')->default(0); // INT DEFAULT 0
$table->date('reservation_date'); // DATE
$table->time('reservation_time'); // TIME
$table->enum('status', ['a', 'b', 'c']); // ENUM
$table->timestamps(); // created_at + updated_at (auto-man

// Foreign key (one line):
$table->foreignId('category_id')->constrained()->onDelete('cascade');
```

Blade Syntax

php

```
{{-- Comment --}}

{{-- Output a variable (escaped) --}}
{{ $variable }}

{{-- Output raw HTML (unescaped, careful!) --}}
{!! $html !!}

{{-- Extending a layout --}}
@extends('layouts.app')

{{-- Filling a slot --}}
@section('title', 'Page Title')

@section('content')
    <p>Page content here</p>
@endsection

{{-- Defining a slot in the layout --}}
@yield('content')
@yield('title', 'Default Title')    {{-- with fallback --}}

{{-- If / else --}}
@if($item->is_available)
    Available
@elseif($item->price > 20)
    Premium
@else
    Unavailable
@endif

{{-- Foreach --}}
@foreach($categories as $category)
    {{ $category->name }}
@endforeach

{{-- Foreach with empty fallback --}}
@forelse($items as $item)
    {{ $item->name }}
@empty
    No items found.
@endforelse
```

```
{{-- For loop --}}
@for($i = 1; $i <= 10; $i++)
    {{ $i }}
@endfor

{{-- CSRF token (required in every POST form) --}}
@csrf

{{-- Method spoofing (for PUT/PATCH/DELETE) --}}
@method('DELETE')

{{-- Validation error for a field --}}
@error('field_name')
    <p>{{ $message }}</p>
@enderror

{{-- Repopulate form after failed validation --}}
value="{{ old('field_name') }}"

{{-- Flash message --}}
@if(session('success'))
    {{ session('success') }}
@endif

{{-- Named route URL --}}
{{ route('menu.index') }}
{{ route('admin.reservations.update', $reservation) }}
```

Routes

php

```
// Basic routes
Route::get('/url', function () { return 'hello'; });
Route::get('/url', [Controller::class, 'method'])->name('route.name');
Route::post('/url', [Controller::class, 'store']);
Route::put('/url/{model}', [Controller::class, 'update']);
Route::delete('/url/{model}', [Controller::class, 'destroy']);

// Route with parameter
Route::get('/menu/{slug}', [MenuController::class, 'category']);
// accessed in controller as: public function category($slug)

// Route model binding (auto-fetches model by ID)
Route::delete('/menu/{menuItem}', [AdminController::class, 'delete']);
// accessed in controller as: public function delete(MenuItem $menuItem)
// Laravel auto-runs MenuItem::find($menuItem) – you get the object directly

// Named route
->name('menu.index')
// used in Blade as: route('menu.index')

// Route group (apply middleware + prefix + name to multiple routes)
Route::middleware(['auth'])->prefix('admin')->name('admin.')->group(function () {
    Route::get('/dashboard', [AdminController::class, 'dashboard'])->name('dash');
    // full route name: admin.dashboard
    // full URL: /admin/dashboard
});
```

Validation Rules

php

```
$request->validate([
    'name'      => 'required|string|max:255',
    'email'     => 'required|email',
    'price'     => 'required|numeric|min:0',
    'date'      => 'required|date|after:today',
    'guests'    => 'required|integer|min:1|max:20',
    'status'    => 'required|in:pending,confirmed,cancelled',
    'optional'  => 'nullable|string|max:500',
]);
// If validation fails: auto-redirects back, flashes errors, repopulates old in
// $validated only contains safe, validated fields – safe to pass to ::create()
```

Middleware

```
php

// Create
php artisan make:middleware AdminMiddleware

// Register alias in bootstrap/app.php
->withMiddleware(function (Middleware $middleware) {
    $middleware->alias([
        'admin' => \App\Http\Middleware\AdminMiddleware::class,
    ]);
})

// Apply to routes
Route::middleware(['auth', 'admin'])->group(function () { ... });

// Or use full class name (always works)
Route::middleware(['auth', \App\Http\Middleware\AdminMiddleware::class])->group
```

Seeders

```
php
```

```
// Create
php artisan make:seeder CategorySeeder

// Run all seeders
php artisan db:seed

// Run specific seeder
php artisan db:seed --class=CategorySeeder

// Wipe and re-seed (fresh start)
php artisan migrate:fresh --seed
```

Artisan Commands Reference

bash

php artisan serve	# start dev server
php artisan migrate	# run migrations
php artisan migrate:rollback	# undo last migration batch
php artisan migrate:fresh	# drop all tables + re-migrate
php artisan migrate:fresh --seed	# drop + re-migrate + seed
php artisan db:seed	# run seeders
php artisan make:model Name	# create model
php artisan make:controller Name	# create controller
php artisan make:migration name	# create migration
php artisan make:seeder Name	# create seeder
php artisan make:middleware Name	# create middleware
php artisan config:clear	# clear config cache
php artisan route:clear	# clear route cache
php artisan cache:clear	# clear app cache
php artisan route:list	# list all routes
php artisan tinker	# interactive console

Starting a New Laravel Project From Scratch

bash

```
# 1. Create project
composer create-project laravel/laravel myproject
cd myproject

# 2. Configure database in .env
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=mydb
DB_USERNAME=root
DB_PASSWORD=root

# 3. Clear config cache
php artisan config:clear

# 4. Run default migrations
php artisan migrate

# 5. Start server
php artisan serve
```

Then for each feature you build:

```
1. php artisan make:migration create_X_table    → define schema
2. php artisan migrate                          → create table
3. php artisan make:model X                    → define model + relationships
4. php artisan make:controller XController     → define methods
5. Add routes to routes/web.php                → map URLs to methods
6. Create blade views in resources/views/      → HTML templates
```

Common Gotchas

Problem	Fix
<code>.env</code> change not working	Run <code>php artisan config:clear</code>
Migration error	Check column order — FK tables must exist before referencing tables
View <code>[x]</code> not found	File must be named <code>x.blade.php</code> , not <code>x.php</code>

Problem	Fix
<code>Mass assignment</code> error	Add column to <code>{fillable}</code> in model
<code>could not find driver</code>	Enable <code>extension=pdo_mysql</code> in <code>php.ini</code>
Old session causing redirect issues	Clear browser cookies for <code>127.0.0.1</code>
MAMP not connecting	Open MAMP and click Start — MySQL must be running
<code>php artisan serve</code> not found	Make sure you're in the project root folder